Manual de Configuración del Sistema de Auditoría de Datos Relacionales

Proyecto ProyectoBD · Motor: PostgreSQL 18 · Versión del documento: 1.0 · 30 de octubre de 2026

1. Introducción

1.1 Propósito

Este manual describe cómo configurar y operar el sistema de auditoría de datos construido sobre PostgreSQL. El sistema registra, a nivel de columna, quién modificó qué dato, cuándo, desde qué dirección IP y con qué valor anterior y nuevo. La auditoría es granular: el administrador decide qué tablas, qué columnas y qué tipos de evento (INSERT, UPDATE, DELETE) se vigilan.

1.2 Alcance

El documento cubre el modelo de configuración, el registro de objetos a auditar, la consulta de la bitácora, la seguridad, el mantenimiento y la solución de problemas. La instalación de la función y de los triggers se detalla en el documento complementario *Procedimiento de Instalación y Despliegue de Triggers*.

1.3 Audiencia

Administradores de base de datos (DBA), auditores internos y desarrolladores responsables de las tablas de negocio.

2. Arquitectura de la solución

El sistema se compone de cuatro capas:

1. **Catálogo de configuración**: tablas que definen qué se audita (`tablas_auditadas`, `columnas_auditadas`, `eventos_auditados`, `configuracion_auditoria`).
2. **Catálogos de contexto**: `usuarios_auditados` y `estaciones_trabajo`, que describen a los usuarios y equipos conocidos.
3. **Motor de captura**: la función de trigger `fn_procesar_auditoria()` y el procedimiento `sp_generar_trigger_tabla()` que instala el trigger en cada tabla.
4. **Repositorio de evidencia**: la tabla `bitacora_auditoria`, donde se guarda cada cambio detectado.

2.1 Flujo de funcionamiento

1. Una aplicación o usuario ejecuta INSERT, UPDATE o DELETE sobre una tabla auditada (por ejemplo `public.clientes`).

2. El trigger `trg_audit_public_clientes` (AFTER, FOR EACH ROW) invoca `fn_procesar_auditoria()`.
3. La función consulta las configuraciones activas que coinciden con la tabla, el esquema y el tipo de operación (`TG_TABLE_NAME`, `TG_TABLE_SCHEMA`, `TG_OP`).
4. Por cada columna configurada extrae el valor antiguo y/o nuevo. En UPDATE solo registra si el valor cambió realmente.
5. Inserta un registro por columna en `bitacora_auditoria` con el usuario de sesión (`session_user`) y la IP del cliente (`inet_client_addr()`).

3. Requisitos previos

Requisito	Detalle
Motor de base de datos	PostgreSQL 18.x (el respaldo fue generado con la versión 18.6 en Ubuntu)
Lenguaje procedural	PL/pgSQL (incluido por defecto)
Rol de instalación	Superusuario o propietario de la base y de las tablas a auditar
Cliente	psql, pgAdmin 4 o equivalente
Codificación	UTF8
Espacio en disco	Planificar crecimiento de `bitacora_auditoria` (ver sección 10)

4. Modelo de datos de configuración

4.1 Descripción de tablas

Tabla	Función	Restricciones clave
`tablas_auditadas`	Tablas habilitadas para auditoría. Campos: `id_tabla`, `nombre_esquema` (por defecto `public`), `nombre_tabla`, `activa`.	UNIQUE (nombre_esquema, nombre_tabla)
`columnas_auditadas`	Columnas vigiladas de cada tabla. Campos: `id_columna`, `id_tabla`, `nombre_columna`, `activa`.	FK a tablas_auditadas con ON DELETE CASCADE; UNIQUE (id_tabla, nombre_columna)
`eventos_auditados`	Catálogo de eventos: INSERT, UPDATE, DELETE.	CHECK de valores permitidos; UNIQUE (tipo_evento)
`configuracion_auditoria`	Cruce columna × evento. Una fila activa significa que ese evento sobre esa columna se registra.	FK a columnas y eventos con ON DELETE CASCADE; UNIQUE (id_columna,

Tabla	Función	Restricciones clave
		id_evento)
bitacora_auditoria	Registro histórico de cambios.	FK a configuracion_auditoria con ON DELETE SET NULL
usuarios_auditados	Catálogo de usuarios con su rol (por defecto OPERADOR).	UNIQUE (username)
estaciones_trabajo	Catálogo de equipos con su IP.	UNIQUE (ip_estacion)

4.2 Estructura de la bitácora

Campo	Tipo	Contenido
id_bitacora	BIGSERIAL	Identificador único del registro
id_config	INT	Configuración que originó el registro (NULL si fue eliminada)
esquema_nombre, tabla_nombre, columna_nombre	VARCHAR	Objeto modificado
tipo_operacion	VARCHAR(10)	INSERT, UPDATE o DELETE
valor_anterior, valor_nuevo	TEXT	Valores convertidos a texto
usuario_bd	VARCHAR(100)	Usuario de sesión que ejecutó la operación
ip_origen	INET	IP del cliente
fecha_hora	TIMESTAMPTZ	Momento del registro (por defecto la hora actual)

4.3 Jerarquía de activación

Para que un cambio se registre deben cumplirse **simultáneamente** estas condiciones:

- `tablas_auditadas.activa = TRUE`
- `columnas_auditadas.activa = TRUE`
- `configuracion_auditoria.activa = TRUE`
- El trigger existe físicamente sobre la tabla (ver documento de despliegue).

Desactivar cualquier nivel suspende la auditoría de ese nivel hacia abajo sin borrar el historial.

5. Configuración paso a paso

5.1 Verificar el catálogo de eventos

Los tres eventos se cargan al instalar el esquema. Verifique:

```
SELECT id_evento, tipo_evento FROM eventos_auditados ORDER BY id_evento;
```

Resultado esperado: 1 = INSERT, 2 = UPDATE, 3 = DELETE. Si falta alguno:

```
INSERT INTO eventos_auditados (tipo_evento)
VALUES ('INSERT'), ('UPDATE'), ('DELETE')
ON CONFLICT (tipo_evento) DO NOTHING;
```

5.2 Registrar una tabla a auditar

```
INSERT INTO tablas_auditadas (nombre_esquema, nombre_tabla)
VALUES ('public', 'clientes');
```

La tabla debe existir previamente. El nombre debe escribirse exactamente como está en el catálogo de PostgreSQL (normalmente en minúsculas).

5.3 Registrar las columnas a vigilar

```
INSERT INTO columnas_auditadas (id_tabla, nombre_columna)
SELECT t.id_tabla, c.col
FROM tablas_auditadas t
CROSS JOIN (VALUES ('correo'), ('telefono')) AS c(col)
WHERE t.nombre_esquema = 'public' AND t.nombre_tabla = 'clientes';
```

Buena práctica: audite solo columnas sensibles o críticas (correos, montos, estados, permisos). Auditar todas las columnas multiplica el volumen de la bitácora.

5.4 Definir qué eventos se registran por columna

Para auditar los tres eventos en todas las columnas de una tabla:

```
INSERT INTO configuracion_auditoria (id_columna, id_evento)
SELECT c.id_columna, e.id_evento
FROM columnas_auditadas c
CROSS JOIN eventos_auditados e
JOIN tablas_auditadas t ON t.id_tabla = c.id_tabla
WHERE t.nombre_esquema = 'public' AND t.nombre_tabla = 'clientes'
ON CONFLICT (id_columna, id_evento) DO NOTHING;
```

Para auditar solo UPDATE y DELETE de una columna concreta:

```
INSERT INTO configuracion_auditoria (id_columna, id_evento)
SELECT c.id_columna, e.id_evento
FROM columnas_auditadas c, eventos_auditados e
WHERE c.nombre_columna = 'correo'
  AND e.tipo_evento IN ('UPDATE', 'DELETE')
ON CONFLICT (id_columna, id_evento) DO NOTHING;
```

5.5 Instalar el trigger en la tabla

```
CALL sp_generar_trigger_tabla('public', 'clientes');
```

El procedimiento crea (o recrea) el trigger `trg_audit_<esquema>_<tabla>`. Para el procedimiento completo, consulte el documento de despliegue.

5.6 Registrar estaciones de trabajo y usuarios (catálogos de contexto)

```
INSERT INTO estaciones_trabajo (nombre_estacion, ip_estacion)
VALUES ('PC-Contabilidad-01', '192.168.1.25');

INSERT INTO usuarios_auditados (username, rol)
VALUES ('jperez', 'OPERADOR'), ('mgarcia', 'ADMIN');
```

Estos catálogos permiten identificar a qué persona y equipo corresponde un `usuario_bd` o una `ip_origen` en la bitácora (ver consulta enriquecida en la sección 7.3).

5.7 Activar y desactivar la auditoría

Objetivo	Sentencia
Pausar toda la tabla	`UPDATE tablas_auditadas SET activa = FALSE WHERE nombre_tabla = 'clientes';`
Pausar una columna	`UPDATE columnas_auditadas SET activa = FALSE WHERE nombre_columna = 'telefono';`
Pausar un evento de una columna	`UPDATE configuracion_auditoria SET activa = FALSE WHERE id_config = 3;`
Reanudar	Repetir con `activa = TRUE`

Los cambios de configuración tienen efecto inmediato en la siguiente operación, sin reinstalar el trigger.

6. Ejemplo completo: auditoría de la tabla clientes

Este ejemplo reproduce la configuración incluida en el respaldo del proyecto: tabla `public.clientes`, columnas `correo` y `telefono`, y los tres eventos.

```
-- 1. Registrar tabla y columnas
INSERT INTO tablas_auditadas (nombre_esquema, nombre_tabla) VALUES
('public','clientes');
INSERT INTO columnas_auditadas (id_tabla, nombre_columna) VALUES (1,'correo'),
(1,'telefono');

-- 2. Habilitar eventos
INSERT INTO configuracion_auditoria (id_columna, id_evento)
SELECT c.id_columna, e.id_evento FROM columnas_auditadas c CROSS JOIN
eventos_auditados e
ON CONFLICT DO NOTHING;

-- 3. Instalar trigger
CALL sp_generar_trigger_tabla('public','clientes');

-- 4. Probar
INSERT INTO clientes (nombre, correo, telefono) VALUES
```

```
('Carlos','carlos@mail.com','5555-1234');
UPDATE clientes SET correo = 'carlos.gomez@mail.com' WHERE nombre = 'Carlos';
DELETE FROM clientes WHERE nombre = 'Carlos';

-- 5. Revisar evidencia
SELECT * FROM bitacora_auditoria ORDER BY id_bitacora;
```

Nota: la columna `clientes.id_cliente` no tiene valor por defecto en el esquema original del dump, por lo que el INSERT de prueba puede requerir indicar `id_cliente` explícitamente o definirlo como SERIAL según su entorno.

7. Consulta y explotación de la bitácora

7.1 Últimos cambios

```
SELECT fecha_hora, usuario_bd, ip_origen, tabla_nombre, columna_nombre,
       tipo_operacion, valor_anterior, valor_nuevo
FROM bitacora_auditoria
ORDER BY fecha_hora DESC
LIMIT 50;
```

7.2 Historial de una columna

```
SELECT fecha_hora, usuario_bd, valor_anterior, valor_nuevo
FROM bitacora_auditoria
WHERE tabla_nombre = 'clientes' AND columna_nombre = 'correo'
ORDER BY fecha_hora;
```

7.3 Enriquecer con catálogos de usuarios y estaciones

```
SELECT b.fecha_hora, b.usuario_bd, u.rol, b.ip_origen, e.nombre_estacion,
       b.tabla_nombre, b.columna_nombre, b.tipo_operacion,
       b.valor_anterior, b.valor_nuevo
FROM bitacora_auditoria b
LEFT JOIN usuarios_auditados u ON u.username = b.usuario_bd
LEFT JOIN estaciones_trabajo e ON e.ip_estacion = b.ip_origen
ORDER BY b.fecha_hora DESC;
```

7.4 Actividad por usuario en un rango de fechas

```
SELECT usuario_bd, tipo_operacion, COUNT(*) AS total
FROM bitacora_auditoria
WHERE fecha_hora >= '2026-10-01' AND fecha_hora < '2026-11-01'
GROUP BY usuario_bd, tipo_operacion
ORDER BY total DESC;
```

8. Reglas de comportamiento de la función de auditoría

Regla	Descripción
Momento de captura	AFTER, por fila: el registro se escribe dentro de la misma transacción que el cambio. Si la transacción se revierte (ROLLBACK), la entrada de bitácora también se revierte.

Regla	Descripción
UPDATE	Solo se registra si `valor_anterior IS DISTINCT FROM valor_nuevo`. Un UPDATE que deja el valor igual no genera registro.
INSERT	`valor_anterior` es NULL.
DELETE	`valor_nuevo` es NULL.
Conversión	Todos los valores se guardan como texto (`::text`).
Usuario	Se toma `session_user` (usuario con el que se inició la sesión, no el rol activo tras SET ROLE).
IP	Se toma `inet_client_addr()`. Es NULL cuando la conexión es por socket Unix local.
Eventos no cubiertos	TRUNCATE y cambios de estructura (DDL) no se auditan.

9. Seguridad y control de acceso

1. **Separación de funciones**: los usuarios de aplicación no deben tener privilegios de UPDATE, DELETE ni TRUNCATE sobre `bitacora_auditoria`.
2. **Rol de auditor**: cree un rol de solo lectura para consulta.
3. **Protección de la configuración**: solo el DBA debe poder modificar las tablas de configuración, porque desactivar la auditoría equivale a ocultar actividad.

```
CREATE ROLE auditor NOLOGIN;
GRANT SELECT ON bitacora_auditoria, tablas_auditadas, columnas_auditadas,
                configuracion_auditoria, eventos_auditados,
                usuarios_auditados, estaciones_trabajo TO auditor;

-- Los usuarios de aplicación no deben tocar la bitácora ni su configuración
REVOKE ALL ON bitacora_auditoria, tablas_auditadas, columnas_auditadas,
              configuracion_auditoria FROM PUBLIC;

-- Permitir que el trigger escriba en la bitácora cuando lo dispara un rol de
aplicación
GRANT INSERT ON bitacora_auditoria TO app_rol;
GRANT SELECT ON tablas_auditadas, columnas_auditadas,
                configuracion_auditoria, eventos_auditados TO app_rol;
GRANT USAGE ON SEQUENCE bitacora_auditoria_id_bitacora_seq TO app_rol;
```

Sustituya `app_rol` por el rol real de su aplicación. Como alternativa más robusta, evalúe declarar la función con `SECURITY DEFINER` y un `SET search_path` fijo, de modo que los roles de aplicación no necesiten permisos directos sobre la bitácora.

4. **Respaldo**: incluya la bitácora en el plan de respaldos con pg_dump o copias físicas.
5. **Datos sensibles**: la bitácora guarda valores en texto claro. Aplique las mismas restricciones de acceso que a la tabla original.

10. Mantenimiento

10.1 Crecimiento y retención

Cada columna modificada genera una fila. Defina una política de retención (por ejemplo, 12 meses en línea y archivo posterior).

```
-- Volumen por tabla
SELECT tabla_nombre, COUNT(*) FROM bitacora_auditoria GROUP BY tabla_nombre ORDER BY
2 DESC;

-- Archivado: mover registros antiguos a una tabla histórica
CREATE TABLE IF NOT EXISTS bitacora_auditoria_hist (LIKE bitacora_auditoria INCLUDING
ALL);
WITH movidos AS (
  DELETE FROM bitacora_auditoria WHERE fecha_hora < now() - interval '12 months'
RETURNING *
)
INSERT INTO bitacora_auditoria_hist SELECT * FROM movidos;
```

Ejecute el archivado con una cuenta privilegiada y deje constancia del procedimiento.

10.2 Índices recomendados

La bitácora no tiene índices más allá de la clave primaria. Para consultas frecuentes:

```
CREATE INDEX idx_bitacora_fecha ON bitacora_auditoria (fecha_hora);
CREATE INDEX idx_bitacora_tabla_col ON bitacora_auditoria (tabla_nombre,
columna_nombre);
CREATE INDEX idx_bitacora_usuario ON bitacora_auditoria (usuario_bd);
```

10.3 Respaldo y restauración

```
pg_dump -h localhost -U postgres -F t -f ProyectoBD.tar "ProyecroBD"
```

10.4 Tareas periódicas

Frecuencia	Tarea
Diaria	Verificar que la bitácora recibe registros esperados
Semanal	Revisar usuarios con mayor actividad de cambios
Mensual	Validar que cada tabla con configuración activa tenga su trigger instalado (ver documento de despliegue, sección de verificación)
Trimestral	Revisar permisos sobre bitácora y configuración; aplicar retención

11. Limitaciones conocidas y diferencias con el modelo ER

Al comparar el diagrama entidad-relación del proyecto (*Auditoría de Datos Relacionales*) con el esquema implementado en el respaldo y en `scripTablas`, se observan diferencias que conviene documentar y, si el alcance lo requiere, corregir:

Elemento del diagrama ER	Implementación actual
`bitacora` con FK a `usuario_auditado` y `estacion_trabajo`	`bitacora_auditoria` guarda `usuario_bd` e `ip_origen` como valores directos, sin FK a los catálogos. La relación es lógica (por coincidencia de username o IP).
`bitacora.id_registro` (identificador de la fila modificada)	No existe. En DELETE y UPDATE no queda registrado el identificador de la fila afectada, solo los valores de la columna.
`bitacora.id_transaccion`	No existe. No es posible agrupar los cambios de una misma transacción.
`tabla_auditada.fecha_creacion` y `creado_por`	No existen en `tablas_auditadas`.
Entidad `evento` con campo `codigo`	Implementada como `eventos_auditados.tipo_evento`.
Nombres en singular (`tabla_auditada`, `columna_auditada`)	Implementados en plural (`tablas_auditadas`, `columnas_auditadas`).
`configuracion_auditoria`	Coincide con el diseño, con restricción UNIQUE adicional (id_columna, id_evento).

Otras limitaciones técnicas:

- La función usa nombres de objetos sin esquema (por ejemplo `bitacora_auditoria`), por lo que depende del `search_path` de la sesión. Se recomienda cualificar con `public.` o fijar `SET search_path` en la función.
- Si la columna configurada no existe en la tabla, la ejecución dinámica falla y **revierte la operación original**. Valide los nombres de columna al registrarlos.
- Se ejecutan dos consultas dinámicas por columna y evento; en tablas de alto volumen considere limitar las columnas auditadas.
- El trigger se instala con los tres eventos aunque la configuración solo active algunos; el filtrado ocurre dentro de la función.

Sugerencia de mejora para `id_registro` (opcional): agregar la columna y poblarla con la clave primaria de la fila, por ejemplo `ALTER TABLE bitacora_auditoria ADD COLUMN id_registro TEXT;` y capturarla en la función.

12. Solución de problemas

Síntoma	Causa probable	Acción
No se genera ningún registro	El trigger no está instalado o hay un nivel de configuración en	Verificar con `SELECT tgname FROM pg_trigger WHERE NOT tgisinternal;` y revisar los tres

Síntoma	Causa probable	Acción
	`activa = FALSE`	niveles de activación
UPDATE no genera registro	El valor de la columna auditada no cambió, o la columna modificada no está configurada	Comportamiento esperado; revisar `columnas_auditadas`
`ip_origen` es NULL	Conexión por socket Unix	Conectarse por TCP (`-h 127.0.0.1`) o aceptar el valor nulo
Error: column ... does not exist al operar la tabla	Nombre de columna mal registrado en `columnas_auditadas`	Corregir el nombre o desactivar la columna
Error de permiso sobre `bitacora_auditoria`	El rol que dispara el trigger no puede insertar en la bitácora	Otorgar INSERT y USAGE de la secuencia, o usar SECURITY DEFINER
Registros con `id_config` NULL	La configuración fue eliminada	Esperado (ON DELETE SET NULL); el historial se conserva
Duplicados al configurar	Violación de UNIQUE	Usar `ON CONFLICT DO NOTHING`

13. Glosario

Término	Significado
Bitácora	Tabla donde se almacenan los eventos de auditoría
Trigger	Disparador que se ejecuta automáticamente ante INSERT, UPDATE o DELETE
TG_OP, TG_TABLE_NAME, TG_TABLE_SCHEMA	Variables de PL/pgSQL con la operación, tabla y esquema que dispararon el trigger
session_user	Usuario con el que se autenticó la sesión
inet_client_addr()	Función que devuelve la IP del cliente conectado
DBA	Administrador de base de datos